

FinSent: Sentiment Analysis per Insight Finanziari Automatizzati

Irene Gaita
Mat: 0522501839
i.gaita@studenti.unisa.it

ABSTRACT

Il progetto *FinSent* propone una pipeline di sentiment analysis finanziaria sviluppata secondo i principi MLOps, con particolare enfasi sull'orchestrazione dei flussi tramite Apache Airflow. Il sistema automatizza le fasi di preprocessing, generazione di embedding, addestramento, valutazione comparativa dei modelli e deployment. Ogni componente è containerizzato tramite Docker e integrato in un'infrastruttura modulare, scalabile e facilmente estendibile.

Il modello selezionato viene esposto tramite una REST API sviluppata con FastAPI, che consente inferenze in tempo reale su frasi testuali. L'architettura include meccanismi di versioning (DVC), gestione automatica del miglior modello e tracciabilità degli esperimenti tramite DAG dedicati. I modelli testati comprendono sia approcci tradizionali (SVM, MLP) sia transformer pre-addestrati (FinBERT, DeBERTa-v3-ft), con risultati di accuratezza fino al 99% in fase di fine-tuning.

FinSent rappresenta un esempio concreto di applicazione di tecniche MLOps a sistemi NLP, con un'infrastruttura orientata al deployment continuo, alla scalabilità e alla gestione automatizzata dell'intero ciclo di vita del modello.

1 INTRODUZIONE

L'analisi automatica del sentiment nelle notizie finanziarie rappresenta un ambito di crescente interesse all'interno della ricerca applicata all'economia computazionale e al machine learning. Le informazioni veicolate attraverso articoli di stampa, report aziendali e comunicati economici possono influenzare in modo significativo le scelte degli investitori e, di conseguenza, l'andamento dei mercati. In tale contesto, lo sviluppo di strumenti in grado di interpretare il contenuto testuale delle notizie dal punto di vista dell'impatto percepito sul valore dei titoli risulta strategico per supportare decisioni d'investimento più rapide, consapevoli e scalabili. Il presente progetto si concentra sulla costruzione e valutazione di una pipeline MLOps per la classificazione automatica del sentiment espresso in notizie riguardanti società quotate nell'indice OMX Helsinki. Il dataset, ottenuto tramite web scraping da fonti in lingua inglese come LexisNexis, è composto da circa 5000 frasi annotate manualmente da un gruppo di 16 valutatori. Ogni frase è stata etichettata come positiva, negativa o neutrale in base al potenziale impatto percepito sul prezzo delle azioni, simulando la prospettiva di un investitore al dettaglio. Questa annotazione manuale conferisce al dataset un'elevata qualità semantica, rendendolo adatto sia all'addestramento sia alla validazione di modelli di apprendimento automatico. Tra le principali sfide affrontate figurano la complessità linguistica dei testi finanziari, spesso caratterizzati da terminologia tecnica e struttura sintattica articolata, nonché l'ambiguità semantica di alcune frasi, interpretabili in modi diversi a seconda del contesto. Un'ulteriore criticità è rappresentata dallo sbilanciamento delle classi, con una marcata prevalenza della

categoria "neutrale", che potrebbe compromettere l'equilibrio predittivo dei modelli. Per arricchire l'analisi è stato inoltre impiegato BERTopic, un algoritmo di topic modeling basato su embeddings semantici, utile a individuare i principali argomenti trattati e ad analizzare la distribuzione del sentiment all'interno dei diversi topic. Il sistema è stato progettato in ottica MLOps, con orchestrazione dei flussi tramite Apache Airflow, tracciamento dei dati tramite DVC, modularizzazione delle fasi di preprocessing, embedding e training, e un servizio di inferenza esposto via FastAPI. La qualità del modello viene valutata mediante metriche di classificazione standard come accuracy, precision, recall e F1-score, tenendo inoltre in considerazione aspetti di scalabilità e efficienza computazionale, con l'obiettivo di ottenere una soluzione applicabile anche in contesti operativi real-time.

2 BUSINESS UNDERSTANDING

2.1 Contesto e problema

Il progetto si inserisce nell'ambito della *Sentiment Analysis* applicata a notizie finanziarie, con un'attenzione particolare alla prospettiva dell'investitore al dettaglio. Questo tipo di analisi risulta fondamentale per migliorare le decisioni di investimento, poiché le emozioni e i giudizi veicolati tramite comunicati stampa, report e articoli economici possono influenzare in modo significativo il comportamento degli operatori finanziari e, di conseguenza, l'andamento dei mercati.

Le notizie, contenenti informazioni economiche e finanziarie, sono state raccolte da fonti in lingua inglese relative a tutte le società quotate nell'OMX Helsinki, uno dei principali indici azionari del mercato finlandese. Il campione di dati proviene da un processo di web scraping automatizzato da un database di notizie come LexisNexis. Ogni frase è stata etichettata come positiva, negativa o neutrale da un gruppo di 16 annotatori, che hanno valutato il contenuto dal punto di vista di un investitore, determinando se una determinata notizia potesse avere un'influenza positiva, negativa o neutra sul prezzo delle azioni. Questa etichettatura risulta essere particolarmente preziosa, poiché il sentiment di ogni notizia è pensato per rispecchiare l'impatto potenziale sui mercati finanziari.

2.2 Obiettivo del progetto

L'obiettivo del progetto è la realizzazione di una pipeline completa per la classificazione automatica del sentiment espresso nei testi finanziari, con un focus su efficienza, scalabilità e automazione. Il sistema dovrà essere in grado di distinguere rapidamente tra notizie a impatto positivo, negativo o neutro, fornendo uno strumento utile per l'analisi in tempo reale e il supporto alle decisioni di investimento.

Il dataset utilizzato è stato concepito come benchmark per la valutazione di modelli di apprendimento automatico applicati alla finanza. Composto da circa 5000 frasi selezionate e annotate con cura,

consente di testare diverse configurazioni di modelli e strategie di preprocessing, nonché di ottimizzare la qualità della classificazione.

2.3 Vincoli e sfide

Il dominio finanziario presenta numerose sfide dal punto di vista linguistico e computazionale. I testi trattati sono spesso caratterizzati da una struttura sintattica complessa, da lessico tecnico-specialistico e da un elevato livello di ambiguità semantica.

Un ulteriore vincolo rilevante è rappresentato dallo sbilanciamento tra le classi del dataset: la classe *neutrale* risulta significativamente più rappresentata rispetto a quelle *positive* e *negative*, introducendo un bias potenziale nei modelli, che tendono a privilegiare la classe dominante.

2.4 Metriche di successo

Il successo del progetto sarà valutato mediante metriche standard di classificazione, in particolare *accuracy*, *precision*, *recall* e *F1-score*. Sarà prioritario ottenere un buon bilanciamento tra precisione e copertura, soprattutto per le classi minoritarie. Inoltre, si terrà conto dell'efficienza computazionale del sistema e della sua capacità di essere integrato in un flusso di lavoro automatico. Il raggiungimento di una soluzione scalabile e applicabile a scenari real-time rappresenta un ulteriore obiettivo di riferimento.

3 DATA UNDERSTANDING

In questa sezione si andrà a discutere della struttura e delle caratteristiche principali del dataset attraverso una prima fase di analisi esplorativa dei dati (EDA). L'obiettivo è comprendere la composizione delle classi di sentiment, la distribuzione delle lunghezze delle frasi e la frequenza delle parole più ricorrenti nel testo originario. Tutte le analisi presentate in questa sezione sono state condotte prima dell'applicazione di qualsiasi tecnica di preprocessing o bilanciamento, in modo da offrire una visione fedele e imparziale della natura grezza dei dati. Verranno inoltre valutati eventuali squilibri tra le classi e la presenza di outlier, aspetti rilevanti per le scelte da adottare nelle successive fasi di pulizia e modellazione del testo.

3.1 Descrizione del dataset originale

Il dataset "FinancialPhraseBank" utilizzato in questo progetto, disponibile pubblicamente su Kaggle ¹ e creato per l'analisi del sentiment delle notizie finanziarie, trae origine da un'attenta raccolta e selezione di articoli finanziari.

La sua creazione, ad opera di Malo et al. come descritto nel lavoro "Good debt or bad debt: Detecting semantic orientations in economic texts," ha impiegato una metodologia precisa.

Inizialmente, gli autori hanno proceduto alla raccolta automatica di notizie tramite uno web scraper, estraendo dati dal database LexisNexis ² per la creazione del corpus. L'attenzione si è concentrata su articoli in lingua inglese relativi alle aziende quotate sull'OMX Helsinki³.

Successivamente, è stata implementata una fase di filtraggio delle frasi. Dalla raccolta iniziale di 10.000 articoli, sono state escluse

le frasi prive di entità lessicali riconosciute, riducendo il corpus a 53.400 frasi.

La fase successiva ha riguardato la classificazione delle frasi. Ogni frase è stata etichettata in base alla rilevanza delle sue sequenze di entità lessicali. Questa annotazione è stata realizzata da un gruppo di 16 annotatori con competenze in ambito aziendale e finanziario, che hanno categorizzato le frasi come positive, negative o neutrali.

Infine, per ottenere un campione rappresentativo e gestibile, è stata effettuata la selezione del campione finale. Dalle 53.400 frasi etichettate, è stato estratto casualmente un sottoinsieme di 4837 frasi, con l'obiettivo di rappresentare l'intero database [3].

La Tabella 1 sintetizza le caratteristiche del dataset.

Numero di frasi	4837
Classi di sentiment	Positive, Negative, Neutral
Fonti delle frasi	Notizie in lingua inglese relative alle aziende quotate in OMX Helsinki
Variabili	Lable, Text

Table 1: Caratteristiche principali del dataset.

3.2 Distribuzione delle Classi di Sentiment

Il dataset è composto da tre classi di sentiment: *positive*, *negative* e *neutral*. Tuttavia, come è possibile osservare nella Tabella 2 e successivamente nella Figura 1, la distribuzione delle classi risulta squilibrata, con una predominanza marcata della classe *neutral*.

Questo aspetto rappresenta un potenziale fattore critico, in quanto un'eccessiva rappresentazione di una classe può introdurre bias nei modelli predittivi, portandoli a preferire la classe maggioritaria indipendentemente dal contenuto testuale. La conseguenza di questo squilibrio è il rischio di compromettere le performance del modello di classificazione, in particolare per quanto riguarda l'accuratezza nella predizione delle classi minoritarie (*positive* e *negative*).

Di conseguenza, è necessario valutare l'impiego di tecniche di bilanciamento — come *undersampling*, *oversampling* o *SMOTE* — che verranno analizzate nei paragrafi successivi.

Classe	Numero di Frasi
Neutral	2879
Positive	1363
Negative	604

Table 2: Distribuzione delle classi di sentiment nel dataset originale.

3.3 Analisi della Lunghezza delle Frasi

Un aspetto cruciale per la comprensione approfondita del dataset testuale è l'analisi della distribuzione della lunghezza delle frasi, misurata in numero di caratteri (inclusa la punteggiatura). Questa metrica fornisce indicazioni preziose sia sulla complessità sintattica intrinseca ai dati, sia per orientare la scelta del modello di elaborazione del linguaggio naturale più appropriato. Come è possibile

¹<https://www.kaggle.com/datasets/ankurzing/sentiment-analysis-for-financial-news/data>

²Fornitore di fonti di informazione legale, governativa, aziendale e high-tech [1].

³Borsa valori di riferimento per la Finlandia [2]

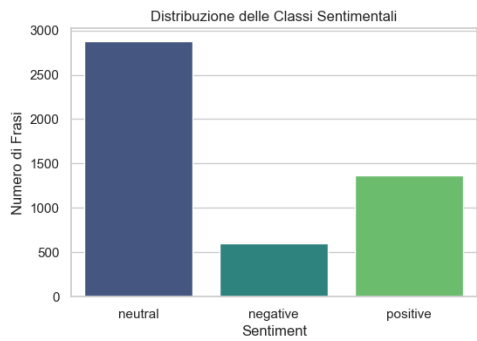


Figure 1: Distribuzione delle classi di sentiment rappresentata graficamente.

osservare nella Tabella 3, l’analisi statistica sulla lunghezza delle frasi rivela una distribuzione moderatamente dispersa, caratterizzata dalla presenza di alcuni valori estremi (outlier) che si discostano significativamente dalla tendenza centrale.

Il valore massimo registrato di 315 caratteri evidenzia la presenza di frasi particolarmente estese, che possono essere interpretate come outlier rispetto alla distribuzione principale. Queste frasi anomale potrebbero potenzialmente all’ introduzione del rumore durante la fase di addestramento del modello, influenzando negativamente la sua capacità di generalizzazione e aumentando il costo computazionale delle operazioni.

Media	128.13 caratteri
Mediana	119 caratteri
Minino	9 caratteri
Massimo	315 caratteri

Table 3: Statistiche sulla lunghezza delle frasi.

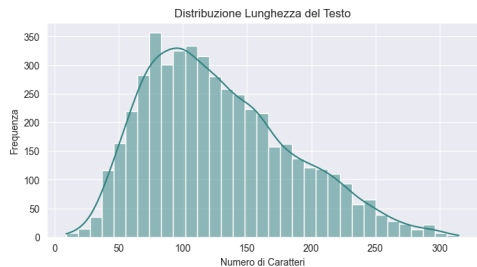


Figure 2: Distribuzione della lunghezza delle frasi nel dataset.

Come si evince dalla Figura 2, la distribuzione della lunghezza delle frasi tende a concentrarsi in un intervallo compreso approssimativamente tra i 100 e i 130 caratteri. Si osserva una frequenza inferiore di frasi significativamente più brevi o, al contrario, eccezionalmente lunghe.

Per approfondire la comprensione della variabilità della lunghezza delle frasi ciascuna delle tre classi di sentiment (*neutral*, *negative*,

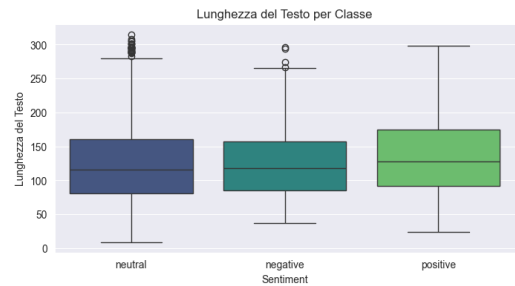


Figure 3: Boxplot delle lunghezze delle frasi per ciascuna classe di sentiment.

Classe	Media	Mediana	Min	Max	Outlier
Neutral	125.07	116	9	315	28
Negative	125.76	118	37	296	4
Positive	135.65	128	24	298	0

Table 4: Statistiche descrittive delle lunghezze delle frasi per classe di sentiment

positive), è stata condotta una analisi attraverso i boxplot presentati in Figura 3.

Dalle statistiche riportate nella Tabella 4, emerge una significativa variabilità nella lunghezza delle frasi tra le diverse classi. In particolare, la classe *neutral* presenta un numero consistente di outlier, mentre la classe *positive* tende a includere frasi mediamente più lunghe rispetto alle altre. Queste peculiarità suggeriscono che, nelle successive fasi di preprocessing e modellazione, potrebbe essere utile tenerne conto per ottimizzare le prestazioni del modello e migliorare la gestione delle differenze stilistiche tra le classi.

3.4 Analisi della frequenza delle parole

È stata effettuata un’analisi della frequenza delle parole presenti nel dataset originale, identificare la presenza di termini ad alta frequenza e valutare il livello di rumore linguistico presente nei dati grezzi.

Come evidenziato nel grafico in Figura 4, le parole più frequenti includono numerosi termini funzionali come articoli (the, a), congiunzioni (and, but), preposizioni (to, of, in) e altre stopword comuni. Sebbene queste parole siano grammaticalmente necessarie, risultano poco informative per il compito di sentiment analysis, in quanto non veicolano contenuto semantico rilevante per distinguere tra sentimenti positivi, negativi o neutri.

3.5 Conclusioni dell’analisi esplorativa

L’analisi preliminare ha evidenziato caratteristiche chiave del dataset che influenzeranno le successive scelte di preprocessing e modellazione. In particolare, lo squilibrio tra le classi, la variabilità nelle lunghezze delle frasi e la predominanza di termini non informativi suggeriscono l’impiego di tecniche di normalizzazione testuale e di strategie mirate per la gestione dello sbilanciamento. Queste osservazioni guideranno la progettazione dei DAG e delle componenti della pipeline MLOps presentate nelle prossime sezioni.

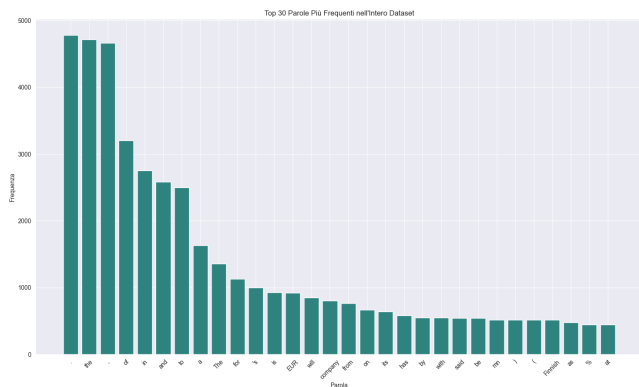


Figure 4: Top 30 parole più frequenti nel dataset originale.

4 DATA PREPARATION

Il dataset utilizzato in questo progetto è composto da notizie finanziarie in lingua inglese, ciascuna associata a un'etichetta di sentiment (*positive*, *neutral*, *negative*). Al fine di rendere il contenuto testuale adatto alle successive fasi di modellazione sono stati sviluppati due moduli di pre-processing strutturati e parametrizzabili, integrati in un DAG di Apache Airflow dedicato alla fase di *data cleaning*.

Sono stati definiti due approcci distinti di normalizzazione del testo:

- un approccio **light**, focalizzato su una pulizia basilare, non invasiva e semanticamente conservativa;
- un approccio **aggressivo**, caratterizzato da una serie di trasformazioni mirate alla riduzione della variabilità linguistica.

Questa distinzione ha permesso di confrontare l'impatto del pre-processing sulla performance dei modelli di classificazione del sentiment e sui task di topic modeling.

4.1 Approccio aggressivo

L'approccio completo prevede una sequenza di trasformazioni linguistiche avanzate, orientate alla standardizzazione semantica e alla riduzione del rumore lessicale. Il modulo è stato implementato come task autonomo nel DAG `dag_preprocessing.py`, in modo da garantire riproducibilità ed eseguibilità automatica. La Tabella 5 riassume i principali passaggi applicati.

I testi con meno di tre parole residue sono stati scartati per evitare input informativi insufficienti. Il risultato finale viene salvato come nuova colonna `cleaned_text` nel dataset processato, versionato con DVC e salvato nel percorso `data/processed/`.

4.2 Approccio leggero (light)

Il pre-processing leggero è stato pensato per algoritmi come BERTopic, in cui una pulizia eccessiva può compromettere la qualità semantica del contenuto. Questo approccio mira a mantenere la struttura sintattica originaria e viene eseguito tramite un modulo dedicato nel DAG `dag_light_preprocessing.py`.

Operazione	Descrizione
<i>Lowercasing</i>	Conversione dell'intero testo in minuscolo.
<i>Separazione lettere-numeri</i>	Espressioni come <code>eur131m</code> diventano <code>eur 131 m</code> .
<i>Normalizzazione intervalli numerici</i>	Intervalli come <code>2009-2012</code> diventano <code>NUM - NUM</code> .
<i>Sostituzione valute</i>	Termini come <code>euro</code> , <code>usd</code> , <code>\$</code> , ecc. vengono sostituiti con il token <code>CUR</code> .
<i>Normalizzazione percentuali</i>	Percentuali come <code>10%</code> o <code>3, 2%</code> diventano <code>NUM_PERCENT</code> .
<i>Sostituzione numeri</i>	Tutti i numeri vengono sostituiti con il token <code>NUM</code> .
<i>Rimozione punteggiatura</i>	Tutti i caratteri non alfanumerici vengono rimossi.
<i>Tokenizzazione</i>	Eseguita con il tokenizer <code>punkt</code> di NLTK.
<i>Rimozione stopwords</i>	Eliminazione delle parole vuote tramite la lista di NLTK.
<i>Lemmatizzazione</i>	Conversione delle parole alla loro forma base con <code>WordNetLemmatizer</code> .

Table 5: Passaggi del pre-processing completo (aggressivo)

Operazione	Descrizione
<i>Rimozione spazi extra</i>	Gli spazi multipli vengono ridotti a uno solo.
<i>Rimozione caratteri non ASCII</i>	Caratteri non appartenenti allo standard ASCII (es. simboli o accenti) vengono eliminati.
<i>Sostituzione simboli valuta</i>	I simboli <code>\$</code> , <code>€</code> , <code>£</code> vengono sostituiti con il token <code>CUR</code> .
<i>Normalizzazione percentuali</i>	Percentuali come <code>5.5%</code> o <code>12%</code> diventano <code>NUM_PERCENT</code> .
<i>Sostituzione numeri</i>	Tutti i numeri interi o decimali vengono sostituiti con il token <code>NUM</code> .

Table 6: Passaggi del pre-processing leggero

Anche in questo caso, frasi troppo brevi sono state filtrate. Il testo risultante è stato conservato come colonna separata nel dataset pre-processato.

5 POST-PROCESSING DATA PROFILING

Al termine del pre-processing, è stata condotta un'analisi esplorativa (EDA) per valutare l'impatto delle trasformazioni applicate al testo e confrontare le principali caratteristiche statistiche del dataset.

5.1 Verifica delle normalizzazioni

Una delle priorità era assicurarsi che la sostituzione dei pattern numerici e simbolici (valute, percentuali, numeri) fosse avvenuta

correttamente. Sono stati selezionati manualmente alcuni esempi di notizie contenenti numeri, valute o percentuali, confrontando il testo originale con quello preprocessato.

Testo originale	Testo preprocessato
“...net sales doubled to EUR131m from EUR76m...”	“net sale doubled CUR 131m CUR 76m”
“...profit margin of 10% - 20% of net sales...”	“profit margin NUM_PERCENT NUM_PERCENT net sale”
“...rose to EUR 13.1 mn from EUR 8.7 mn...”	“rose CUR NUM mn CUR NUM mn...”

Table 7: Esempi di sostituzioni corrette nel testo

Come evidenziato in Tabella 7, i token NUM, CUR e NUM_PERCENT sono stati correttamente applicati al posto dei corrispondenti elementi originali.

5.2 Distribuzione della lunghezza dei testi

Per valutare l’omogeneità dei testi preprocessati, è stata analizzata la distribuzione della lunghezza (in caratteri) dei contenuti nella colonna cleaned_text. La Figura 5 mostra l’istogramma relativo.

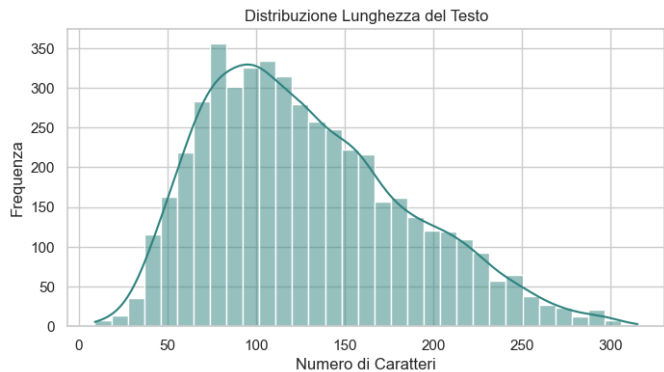


Figure 5: Distribuzione della lunghezza dei testi dopo il pre-processing

Le statistiche principali osservate sulla lunghezza dei testi sono:

- **Media:** 95.02 caratteri
- **Mediana:** 87 caratteri
- **Minimo:** 13 caratteri
- **Massimo:** 291 caratteri

In Tabella 3 sono presenti le stesse statistiche, ma prima del pre-processing.

Dal confronto emerge una riduzione complessiva della lunghezza dei testi:

- la **media** è diminuita di circa il 26%;
- anche la **mediana** è calata, segno che la trasformazione ha agito in modo diffuso su tutto il corpus;
- il **minimo** è aumentato, poiché sono stati rimossi i testi troppo brevi dopo il pre-processing;

- il **massimo** è leggermente diminuito, ma sono stati preservati anche i testi più lunghi.

Questa riduzione è giustificata dall’eliminazione di stopwords, punteggiatura, simboli e dalla normalizzazione di entità numeriche e valutarie, che tendono a comprimere la rappresentazione testuale mantenendo però l’informazione semantica utile al modello.

5.2.1 Analisi per classe di sentiment. L’effetto del pre-processing è stato analizzato separatamente per ciascuna classe (*neutral*, *negative*, *positive*). La Tabella 8 riassume i dati più rilevanti.

Classe	Media originale (parole)	Media preprocessato (parole)
Neutral	22	11
Negative	24	12
Positive	25	13

Table 8: Confronto lunghezza media delle frasi per classe (prima/dopo)

Classe Neutral. La lunghezza media delle frasi si riduce da 22 a 11 parole. Il numero di outlier resta costante (27), ma le frasi risultano più compatte e meno ridondanti.

Classe Negative. Le frasi passano in media da 24 a 12 parole. Gli outlier si riducono sensibilmente, confermando una maggiore regolarità. Frasi molto lunghe (oltre 50 parole) vengono eliminate o contratte.

Classe Positive. Anche qui la lunghezza media si dimezza, da 25 a 13 parole. Il preprocessing preserva comunque frasi fino a 32 parole, mantenendo quindi una buona informatività.

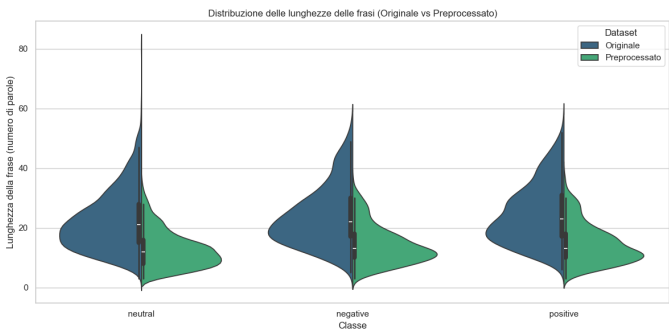


Figure 6: Violin plot: distribuzione frasi - originale vs preprocessato

La Figura 6 mostra graficamente gli effetti del pre-processing sulle singole classi confrontando la distribuzione della lunghezza delle frasi prima e dopo la pulizia del corpus.

Considerazioni. Il pre-processing ha ridotto la lunghezza media in tutte le classi, eliminando parole ridondanti o di scarso valore informativo (es. stopwords). Ciò favorisce modelli NLP più efficienti e meno sensibili al rumore testuale. La struttura sintattica

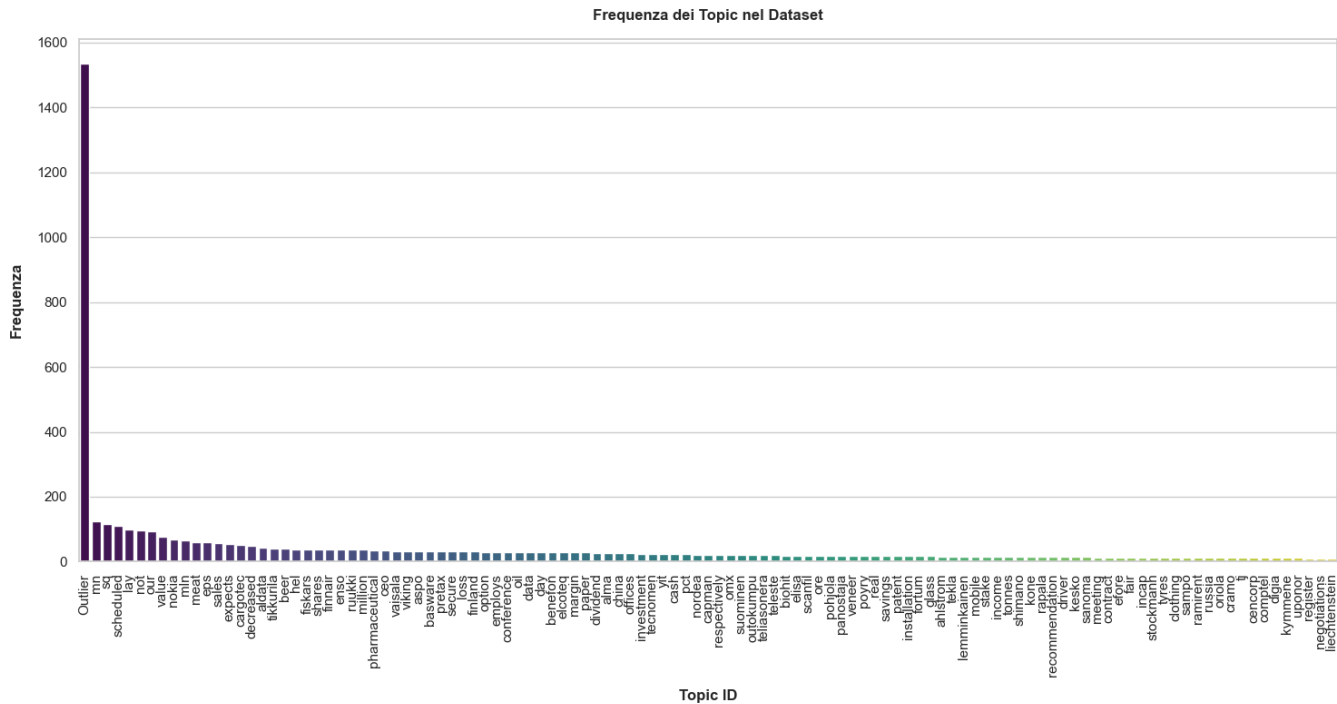


Figure 8: Distribuzione dei topic del dataset con pre-processing completo

tematici. I topic dominati da sentiment positivo o negativo risultano invece meno numerosi, ma potenzialmente più specifici e legati a eventi rilevanti, come comunicazioni sui risultati trimestrali o modifiche strategiche aziendali. L’esistenza di topic mixed evidenzia infine la presenza di ambiguità o sfumature nei testi, che potrebbero rappresentare un’importante area di miglioramento per i modelli di classificazione automatica.

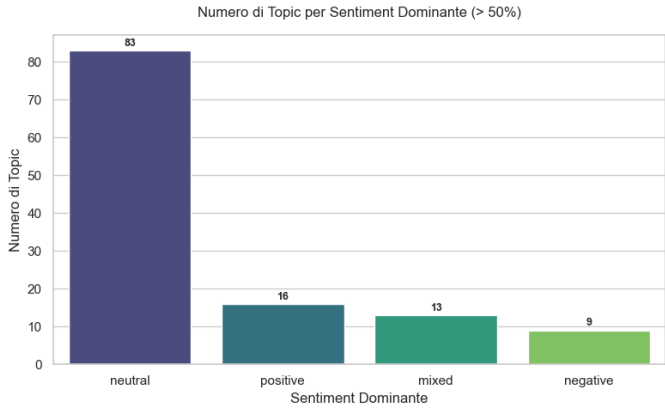


Figure 9: Distribuzione percentuale delle etichette di sentiment per ogni topic.

È stata esaminata la natura dei documenti assegnati al cluster di outlier, ovvero quei testi che BERTopic non è riuscito ad associare in modo coerente a uno dei topic principali. L’obiettivo era verificare

se tali frasi presentassero caratteristiche linguistiche differenti, in particolare in termini di lunghezza del testo. L’analisi, basata su un boxplot comparativo in Figura 10, ha mostrato che non vi sono differenze significative tra gli outlier e i non-outlier in termini di lunghezza media del testo (13.49 vs 13.46 parole). Questo suggerisce che l’isolamento di tali frasi non è attribuibile a una struttura testuale anomala, ma piuttosto a contenuti semantici poco affini ai topic principali o a una maggiore ambiguità lessicale.

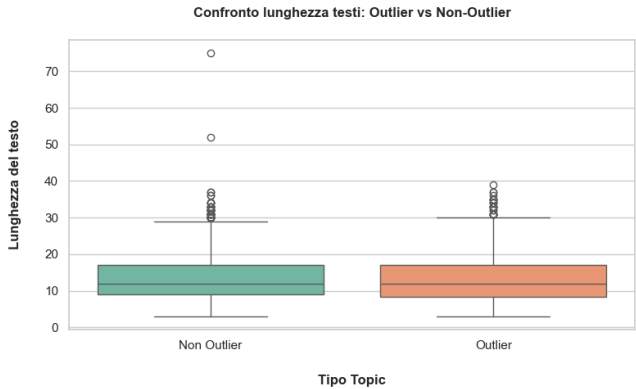


Figure 10: Boxplot comparativo delle lunghezze delle frasi appartenenti ai cluster Outlier e Non Outlier.

Per comprendere meglio la natura semantica dei documenti classificati come outlier (topic -1), è stata effettuata un’analisi lessicale

delle parole più frequenti visibile in Figura 11, confrontando il vocabolario dei testi outlier con quello dei non-outlier. Entrambi i gruppi presentano alcune parole comuni, come "num", "cur", "company", "said" e "finnish", ma con frequenze differenti: ad esempio, la parola "num" compare 1489 volte tra gli outlier e 4220 tra i non-outlier, mantenendo tuttavia una posizione dominante in entrambi i casi.

Tuttavia, nei testi assegnati a topic coerenti (non-outlier), emergono con maggiore chiarezza parole legate a concetti economico-finanziari specifici, come "sale", "profit", "share", "operating", "period" e "market". Al contrario, i topic outlier mostrano una maggiore ricorrenza di termini generici ("business", "new", "group", "oyj"), che potrebbero indicare una mancanza di contesto o una struttura informativa più debole. Questo risultato suggerisce che gli outlier non sono solo semanticamente eterogenei, ma potenzialmente anche meno informativi o contestualmente ambigui, rendendo più difficile per l'algoritmo associarli a un topic coerente.

6.2 Analisi spaziale degli embedding

Al fine di indagare la struttura semantica del corpus da una prospettiva geometrica, è stata condotta un'analisi spaziale degli embedding testuali. A tale scopo, ciascuna frase preprocessata è stata convertita in un vettore numerico attraverso il modello all-MiniLM-L6-v2 della libreria SentenceTransformer, che fornisce una rappresentazione densa e semantica del testo basata su transformer leggeri.

Per rendere visibile la distribuzione dei documenti nel piano, gli embedding sono stati proiettati in due dimensioni tramite la tecnica UMAP (Uniform Manifold Approximation and Projection), utilizzando la metrica del coseno e mantenendo `n_neighbors=15`. Il risultato è stato visualizzato mediante uno scatterplot bidimensionale, in cui ciascun punto rappresenta una frase del corpus, colorata in base alla sua appartenenza al topic outlier (in rosso) o a uno dei topic coerenti (in verde).

Dal grafico in Figura ?? emerge che i documenti classificati come outlier risultano generalmente più dispersi e isolati rispetto ai punti associati a topic ben definiti, che tendono a formare agglomerati compatti. Sebbene siano presenti alcuni piccoli gruppi di embedding outlier vicini tra loro, la loro numerosità risulta insufficiente per generare un topic coerente. Questo comportamento conferma che l'assegnazione a un topic outlier non è riconducibile né alla lunghezza del testo né alla frequenza di termini specifici, ma piuttosto alla posizione nel contesto semantico globale del corpus. In altre parole, la classificazione come outlier riflette una distanza semantica rispetto ai nuclei tematici dominanti, evidenziando frasi che non si integrano facilmente in cluster concettualmente omogenei.

A completamento dell'analisi semantica, è stata visualizzata la distribuzione spaziale dei topic generati da BERTopic proiettando gli embedding testuali nel piano bidimensionale tramite UMAP. A differenza della precedente visualizzazione, in questo caso ciascun punto è stato colorato in base al topic di appartenenza, al fine di analizzare la coerenza interna e la separazione tra i diversi cluster semantici.

Dal grafico in Figura 13 emergono alcune considerazioni interessanti. In primo luogo, i colori tendono a formare regioni compatte

e ben separate, soprattutto lungo i margini della mappa. Questo comportamento suggerisce che BERTopic ha segmentato il corpus in maniera coerente dal punto di vista semantico, assegnando molte frasi a topic caratterizzati da un'alta coesione interna. Tali aree possono essere interpretate come topic forti, ovvero gruppi di documenti che condividono un contenuto tematico fortemente omogeneo.

Tuttavia, nella zona centrale della mappa si osserva la presenza di regioni meno definite, in cui i topic si sovrappongono parzialmente o si collocano a distanza ravvicinata. Questo fenomeno è tipico di frasi che trattano argomenti simili o contigui, e riflette una vicinanza semantica tra topic differenti. In prospettiva, tale informazione potrebbe essere sfruttata per unificare automaticamente i topic semanticamente adiacenti, migliorando la sintesi e riducendo la frammentazione dell'output.

Per valutare la coerenza emotiva dei topic identificati, è stata condotta un'analisi della polarizzazione del sentiment all'interno di ciascun cluster. È stata calcolata, per ogni topic, la distribuzione percentuale delle etichette di sentiment (positivo, negativo, neutrale) e determinato il sentiment dominante, ovvero la classe con la frequenza più alta. In seguito, sono stati selezionati i topic in cui una singola etichetta superava la soglia dell'80%, indicativi di un'elevata omogeneità interna in termini di polarità emotiva.

I risultati, visualizzati tramite una heatmap visibile in Figura 14 e sintetizzati nella Tabella 9 ordinata per grado di polarizzazione, mostrano che diversi topic presentano una chiara dominante emotiva, con percentuali superiori anche al 90%. Questi topic possono essere interpretati come cluster tematici coerenti non solo semanticamente, ma anche sul piano del sentiment, e possono costituire una base solida per attività di classificazione supervisionata o generazione di etichette automatiche.

Tale polarizzazione risulta particolarmente evidente nei topic associati a eventi fortemente positivi (es. acquisizioni, utili superiori alle attese, annunci di dividendi) o negativi (es. licenziamenti, perdite operative, revisioni al ribasso). Viceversa, i topic più generici o descrittivi tendono a mostrare una distribuzione più bilanciata delle etichette, riflettendo la natura neutra dell'informazione contenuta.

Topic	Sentimento Dominante	Percentuale Dominante	Numero occorrenze
36	Neutrale	100.00%	30
105	Neutrale	100.00%	13
77	Neutrale	100.00%	17
49	Positivo	100.00%	26

Table 9: Topic con polarizzazione sentimentale dominante.

A completamento dell'analisi sulla polarizzazione dei topic, è stato effettuato un confronto lessicale tra alcuni dei cluster con sentiment dominante ben definito. In particolare, sono stati analizzati i topic 36 (neutrale), 41 (negativo) e 49 (positivo), con l'obiettivo di identificare eventuali parole in comune tra gruppi apparentemente opposti sul piano emotivo.

I risultati mostrano che, nonostante la polarità differente, esistono sovrapposizioni lessicali significative. Ad esempio, il topic

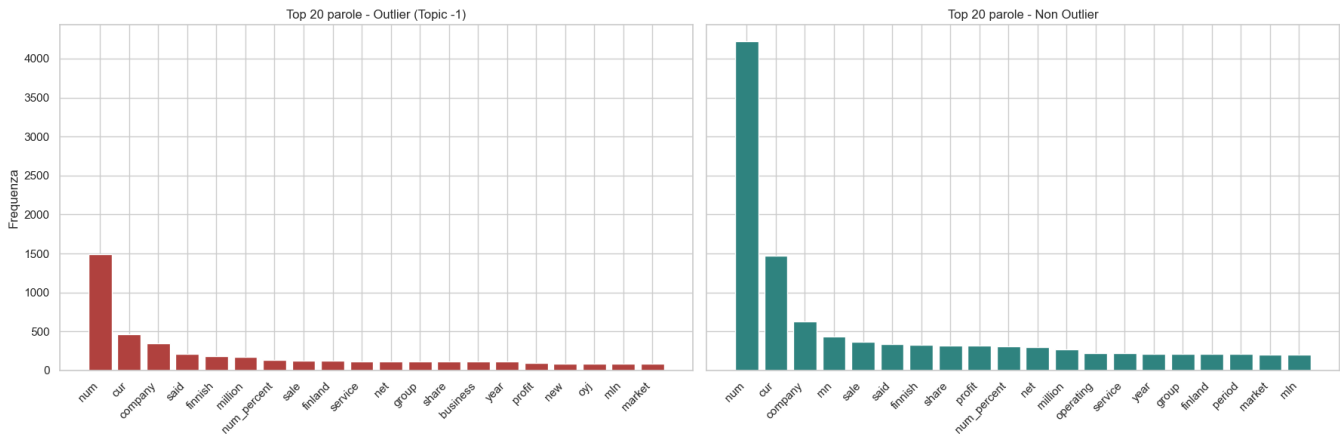


Figure 11: Distribuzione delle parole del vocabolario dei testi outlier con quello dei non-outlier.

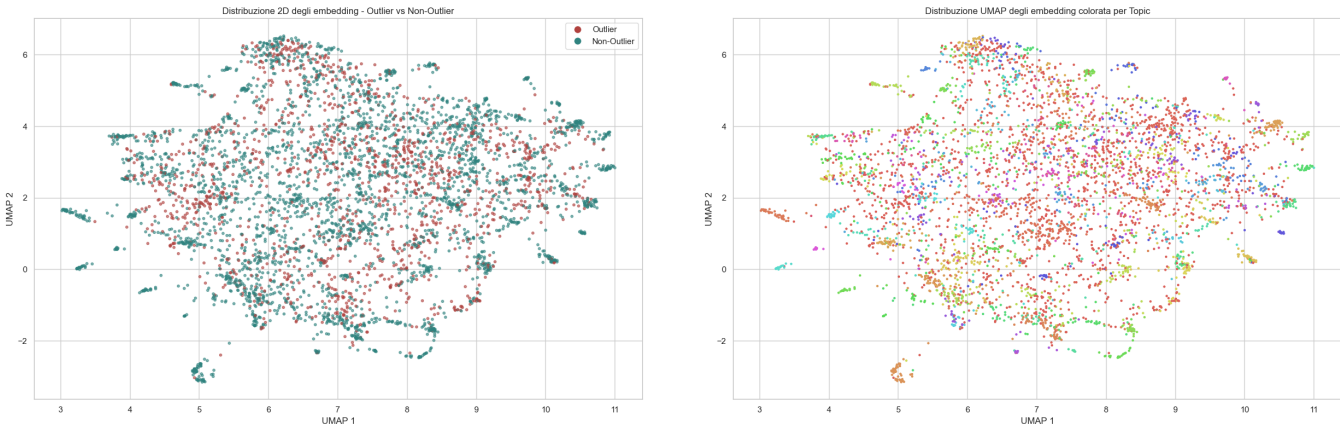


Figure 12: Distribuzione UMAP – outlier vs non-outlier.

Figure 13: Distribuzione UMAP – frasi colorate per topic.

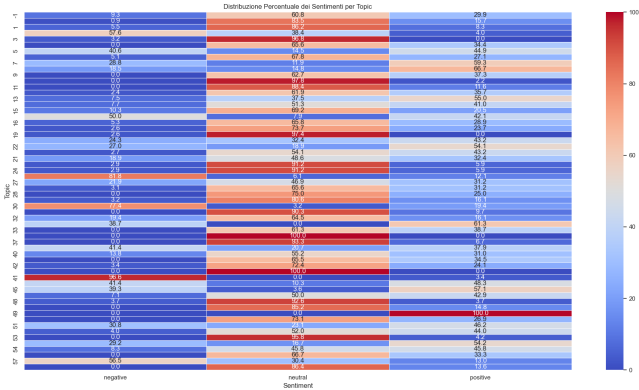


Figure 14: Heatmap della distribuzione percentuale delle etichette di sentiment per ogni topic.

termini come quarter, net, profit, mn, operating e income, tutti appartenenti al vocabolario tecnico-finanziario. Queste parole, pur essendo comuni nei resoconti economici, possono assumere connotazioni diverse a seconda del contesto narrativo e delle strutture sintattiche circostanti.

Anche tra i topic neutrale (36) e positivo (49), sono presenti 5 termini ricorrenti, come net, sale, group e NUM. Il topic neutrale condivide invece solo 4 parole con quello negativo. Questo conferma che la co-occorrenza di termini tecnici non implica necessariamente una coerenza nel sentiment, sottolineando la necessità di modelli in grado di interpretare non solo il lessico, ma anche le relazioni semantiche più profonde e il contesto narrativo.

6.3 Generazione Embedding

La generazione degli embedding è stata orchestrata tramite un DAG dedicato in Apache Airflow, denominato dag_generate_embeddings. Questo workflow si compone di due task sequenziali - generate_word2vec_embeddings e generate_sbert_embeddings

41 (negativo) condivide 16 parole con il topic 49 (positivo), tra cui

— implementati come `PythonOperator`. La logica prevede la generazione dei vettori `Word2Vec` seguita dalla generazione degli embedding SBERT, consentendo così di mantenere un ordine esplicito e riproducibile delle operazioni.

Entrambi i task operano sullo stesso dataset, preprocessato in modalità aggressiva.

Task Word2Vec. Il primo task del DAG carica un modello preaddestrato `glove-wiki-gigaword-50` tramite la libreria `gensim.downloader`, selezionato per la sua leggerezza (50 dimensioni) e per la buona copertura lessicale in contesto economico. Ogni frase viene trasformata in un vettore ottenuto calcolando la media dei vettori di tutte le parole riconosciute nel modello. Se una frase non contiene parole nel vocabolario, viene assegnato un vettore nullo.

Task SBERT. Il secondo task utilizza il modello `all-MiniLM-L6-v2` della libreria `sentence-transformers`, scelto per il suo buon trade-off tra accuratezza e efficienza computazionale. A differenza di `Word2Vec`, SBERT genera embedding direttamente a livello di frase, sfruttando architetture transformer ottimizzate per il sentence-level encoding.

Tecniche alternative sperimentate. Durante la fase di sperimentazione, sono stati testati anche modelli alternativi per la generazione degli embedding, tra cui `bert-base-uncased` e `FinBERT`, specificamente addestrato su testi di dominio finanziario. `FinBERT` ha mostrato le performance migliori in termini di coerenza semantica e capacità discriminativa nei task di sentiment analysis. Tuttavia, a causa dell'elevato costo computazionale e dei requisiti infrastrutturali non compatibili con il deployment previsto, si è optato per i modelli `all-MiniLM-L6-v2` e `glove-wiki-gigaword-50`, che garantiscono un miglior compromesso tra accuratezza ed efficienza all'interno del DAG di produzione.

6.4 Salvataggio degli embedding

Tutti gli embedding sono stati salvati in formato `.npy` (NumPy array binari). Questa scelta è motivata da esigenze di efficienza:

- il formato binario è molto più veloce da leggere e scrivere rispetto a `.csv`;
- consente il salvataggio diretto di array multidimensionali senza conversioni;
- evita problemi legati alla formattazione dei valori numerici (es. separatori decimali, encoding).

Gli embedding così salvati sono stati successivamente ricaricati nella fase di addestramento dei modelli di classificazione.

6.5 Caratteristiche comparate delle tecniche

La Tabella 10 riassume le principali caratteristiche delle tecniche di embedding utilizzate.

Tecnica	Dim. Vettore	Livello	Tipo
Word2Vec	300	Parola / Frase (media)	Denso
BERT	768	Frase (token [CLS])	Denso
SBERT	384	Frase	Denso
FinBERT	768	Frase (token [CLS])	Denso

Table 10: Confronto tra tecniche di embedding

- **Livello:** indica se l'embedding è ottenuto parola per parola o sull'intera frase.
- **Tipo:** tutti i metodi producono array numerici densi (*dense vectors*), ideali per modelli MLP o SVM.

7 MODELLI BASELINE E ADDESTRAMENTO

Dopo aver generato gli embedding testuali, sono stati addestrati diversi modelli di classificazione supervisionata. Lo scopo non è ottenere prestazioni ottimali, ma stabilire dei **modelli baseline** con cui confrontare approcci più sofisticati nelle fasi successive. Questi modelli offrono un primo riferimento per valutare l'efficacia delle rappresentazioni testuali e delle feature disponibili.

7.1 Data Preparation

Scelta del dataset. In fase iniziale viene verificata la presenza del dataset con preprocessing **light** o **aggressivo**, e caricato dinamicamente in base alla sperimentazione da eseguire.

Encoding della label. La colonna `label` contiene valori testuali (*positive, neutral, negative*), che devono essere convertiti in valori numerici per poter essere interpretati dai modelli. A questo scopo è stato utilizzato un `LabelEncoder` di `sklearn`, che assegna un intero univoco a ciascuna classe.

Inclusione della feature topic. Il training dei modelli è stato eseguito sia con che senza questa feature, per verificare se l'aggiunta del topic migliorasse la capacità predittiva del modello. I topic sono stati rappresentati come numeri interi e concatenati agli embedding solo quando previsto.

Embedding e split dei dati. Gli embedding salvati in formato `.npy` sono stati caricati in memoria e, se necessario, arricchiti con la colonna `topic`. Successivamente i dati sono stati suddivisi in training e validation set tramite `train_test_split` con una suddivisione standard 80/20.

8 MODELING

Sono stati selezionati quattro modelli di classificazione noti per le loro buone prestazioni in contesti baseline:

- **Logistic Regression (LogReg)**
- **Support Vector Machine (SVM)**
- **Random Forest (RF)**
- **Multilayer Perceptron (MLP)**

8.1 Combinazioni sperimentali

Ciascun modello è stato testato su diverse configurazioni di embedding, includendo sia `Word2Vec` che SBERT, con e senza bilanciamento delle classi. In totale sono state valutate quattro combinazioni

principali, integrate nei DAG di training, mentre ulteriori varianti (come BERT e FinBERT) sono state esplorate in fase sperimentale al di fuori del workflow automatizzato.

8.2 Standardizzazione delle feature

Per modelli sensibili alla scala delle feature (come LOGREG, SVM e MLP), è stata applicata la **standardizzazione** tramite StandardScaler. Questa trasformazione porta ogni feature a media zero e varianza unitaria, condizione che migliora la convergenza dell’ottimizzazione e stabilizza il training.

8.3 Valutazione e confronto dei modelli

Ogni modello è stato valutato tramite le seguenti metriche:

- Accuracy
- Precision
- Recall
- F1-score

I risultati sono stati raccolti in un unico DataFrame e ordinati in modo decrescente rispetto all’**accuracy**, per identificare il miglior classificatore.

La Figura 15 mostra un confronto tra i modelli baseline in termini di **accuracy**, includendo diverse combinazioni di embedding e l’eventuale presenza della feature topic.

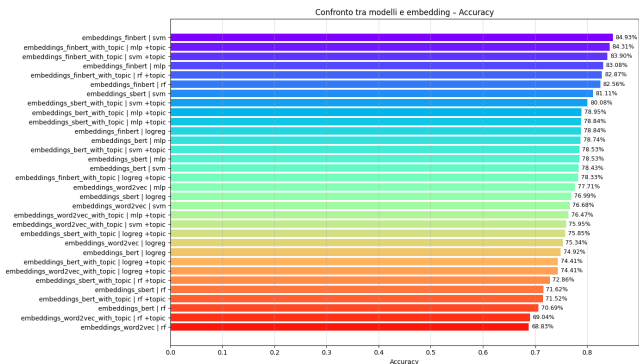


Figure 15: Confronto tra modelli e tecniche di embedding (ordinati per accuracy)

Dall’analisi emergono alcune osservazioni interessanti:

- Il **modello con le migliori performance** è un SVM addestrato su embedding FinBERT **senza** l’aggiunta della feature topic, con un’accuracy pari all’84.93%.
- FinBERT si dimostra complessivamente l’embedding più efficace, con ottimi risultati anche in combinazione con MLP, Random Forest e con la feature topic.
- L’aggiunta del topic non garantisce un miglioramento sistematico: in alcuni casi migliora leggermente la performance (es. MLP con FinBERT), ma in altri può addirittura ridurre la precisione, indicando che il topic potrebbe introdurre rumore o ridondanza.
- Modelli con embedding Word2Vec risultano significativamente meno efficaci, anche con l’aggiunta del topic, suggerendo che tecniche più dense e contestuali sono preferibili in questo task.

8.4 Miglior modello e matrice di confusione

Il modello con la migliore accuracy è stato ricalibrato e rieseguito sull’intero dataset di test, al fine di generare la relativa **matrice di confusione**, utile a visualizzare gli errori commessi tra le classi.

La Figura 16 mostra la matrice di confusione ottenuta rieseguendo il miglior modello (SVM + FinBERT) sul test set.

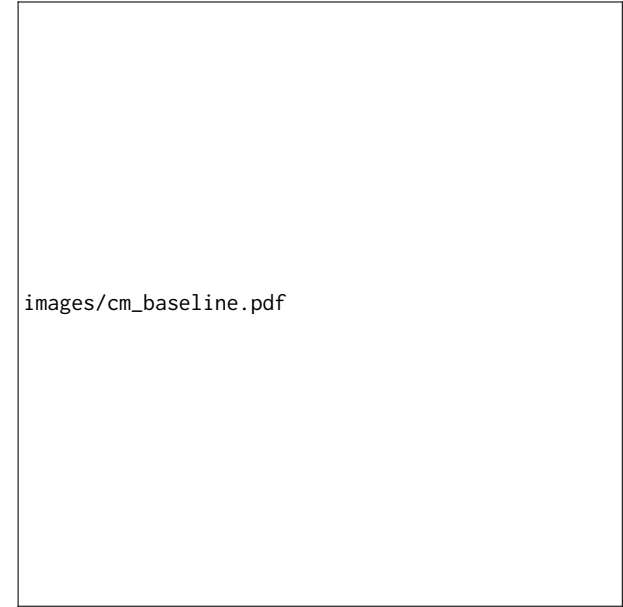


Figure 16: Matrice di confusione per il miglior modello (SVM + FinBERT)

- La classe **neutral** è la più facilmente riconoscibile, con 534 istanze correttamente classificate. Tuttavia, presenta anche sovrapposizioni con entrambe le altre classi.
- Le classi **positive** e **negative** vengono in parte confuse con la **neutral**, in particolare le frasi positive (75 errori).
- Gli errori *positivi* ↔ *negativi* sono molto rari, indicando che il modello riesce a distinguere bene sentimenti opposti, mentre fatica a separare sentimenti chiari da quelli più sfumati o descrittivi.

Nota bene: i risultati riportati sono stati ottenuti utilizzando il dataset sottoposto a **pre-processing leggero**. La medesima pipeline è stata successivamente applicata anche al dataset con **pre-processing aggressivo**, ma in questo caso le prestazioni sono risultate inferiori. Pertanto, nel presente documento vengono mostrati esclusivamente i risultati relativi al **pre-processing light**.

Conclusioni preliminari. FinBERT si dimostra particolarmente efficace in un contesto finanziario, grazie alla sua specializzazione sul dominio. I modelli baseline, in particolare SVM e MLP, offrono prestazioni già solide e rappresentano una base affidabile per confronti con modelli più avanzati nelle fasi successive del progetto. L’inclusione del topic come variabile aggiuntiva ha prodotto risultati contrastanti: in alcuni casi ha migliorato lievemente le performance, ma in molti altri non ha apportato benefici significativi,

arrivando persino a peggiorare le metriche rispetto allo stesso modello addestrato senza topic. Questo suggerisce che l'informazione tematica, almeno nella forma in cui è stata incorporata, non rappresenta un forte discriminatore per il compito di sentiment analysis.

8.5 Pipeline automatizzate vs. risultati sperimentali

Nel progetto sono state implementate e orchestrate quattro pipeline di training automatizzate tramite DAG in Apache Airflow. Ciascuna combinava un tipo di embedding e una strategia di bilanciamento:

- Word2Vec **con** bilanciamento delle classi
- Word2Vec **senza** bilanciamento
- SBERT **con** bilanciamento
- SBERT **senza** bilanciamento

Queste configurazioni sono state scelte per il loro buon compromesso tra semplicità, efficienza computazionale e riproducibilità all'interno del contesto di produzione previsto.

Risultati sperimentali superiori (non inclusi nei DAG). Parallelamente, sono state condotte sperimentazioni aggiuntive con modelli e embedding non inclusi nei DAG, tra cui Bert e FinBERT, specificamente addestrato su testi di dominio finanziario.

FinBERT ha fornito i risultati migliori in assoluto, raggiungendo un'accuracy dell'84.93% in combinazione con un classificatore SVM, **senza l'aggiunta della feature** topic. Questo lo rende il modello più performante nel contesto del task di sentiment analysis considerato.

Esclusione di FinBERT dai DAG. Nonostante l'elevata accuratezza, FinBERT non è stato incluso nel workflow automatizzato per motivi progettuali. In particolare:

- **Costo computazionale elevato:** i tempi di inferenza e i requisiti hardware di FinBERT risultano poco compatibili con l'ambiente di produzione previsto.
- **Sostenibilità operativa:** i modelli scelti per i DAG dovevano garantire leggerezza, scalabilità e semplicità di deploy.
- **Trade-off consapevole:** SBERT (all-MiniLM-L6-v2) offre performance competitive con un ingombro computazionale molto inferiore, rendendolo più adatto per un'integrazione continua e automatizzata.

Questa distinzione tra **sperimentazione libera** e **automazione stabile** riflette un principio chiave del progetto: massimizzare la qualità del modello compatibilmente con i vincoli tecnici e operativi reali.

8.6 DAG di confronto automatico tra esperimenti

Per garantire una valutazione sistematica e riproducibile delle diverse configurazioni di training, è stato sviluppato un DAG dedicato alla fase di comparazione. Questo workflow analizza automaticamente tutte le directory di modelli salvate, escludendo quelle etichettate come latest, ed estrae i parametri di configurazione (embedding, balancing, ecc.) insieme alle metriche di performance salvate durante il training.

Le informazioni raccolte includono, per ciascun esperimento:

- Nome dell'esperimento e timestamp di esecuzione

- Tipo di embedding e strategia di bilanciamento
- Accuracy complessiva
- F1-score per ciascuna classe (positive, neutral, negative)

Tutti i risultati vengono ordinati in base all'accuracy e salvati in un file CSV centralizzato, utilizzabile per analisi successive o per generare visualizzazioni aggregate. Questo approccio consente di mantenere uno storico coerente degli esperimenti effettuati e facilita il confronto oggettivo tra modelli.

8.7 DAG per l'aggiornamento automatico del miglior modello

A completamento del ciclo di training e valutazione, è stato implementato un DAG con lo scopo di automatizzare la selezione e l'aggiornamento del modello con le migliori performance. Il DAG è composto da due task sequenziali:

- **compare_experiments:** legge tutte le directory di esperimenti completati (escludendo quelle latest), estrae i relativi file di configurazione e metriche, e genera una tabella comparativa ordinata per accuracy, salvata come `comparison_table.csv`.
- **update_best_model_latest:** identifica l'esperimento con la migliore accuracy e aggiorna la cartella `best_model_latest` copiando al suo interno i file chiave del modello vincente (`model.pkl`, `metrics.json`, `config.json`).

Questa procedura consente di mantenere sempre disponibile un modello "ultimo e migliore", pronto per l'inferenza o il deployment, senza necessità di intervento manuale. Il DAG è pensato per essere eseguito on-demand oppure integrato in pipeline più ampie, rafforzando l'approccio MLOps del progetto.

9 MODELLI PRE-ADDESTRATI: INFERENZA E FINE-TUNING

Questa sezione descrive una fase sperimentale condotta al di fuori della pipeline automatizzata implementata tramite DAG. L'obiettivo è stato valutare l'efficacia di modelli transformer pre-addestrati, sia in inferenza diretta che dopo fine-tuning, rispetto ai modelli tradizionali basati su embedding statici.

Questa parte del progetto è suddivisa in due fasi: una prima di **inferenza diretta** (zero-shot) e una successiva di **fine-tuning supervisionato** sul dataset specifico.

9.1 Inferenza con modelli pre-addestrati

Sono stati testati cinque modelli pre-addestrati disponibili tramite la libreria Transformers di HuggingFace. L'obiettivo era verificare le performance **senza alcun riaddestramento**, sfruttando direttamente le conoscenze linguistiche già acquisite.

Motivazione. L'inferenza con modelli pre-addestrati consente di ottenere predizioni a partire dal **testo raw originale**, senza necessità di calcolare manualmente gli embedding. Questo è possibile perché tali modelli incorporano internamente tokenizer, layers di embedding e testate di classificazione. In altre parole, il modello riceve la frase testuale completa ed elabora automaticamente la rappresentazione vettoriale necessaria per classificare il sentiment.

Valutazione. Per garantire coerenza con gli esperimenti precedenti, è stato utilizzato lo stesso test set dei modelli baseline. Ogni modello è stato valutato tramite le metriche standard: **accuracy**, **precision**, **recall**, **F1-score**. I risultati sono stati raccolti in un DataFrame ordinato per accuracy e visualizzati nella Figura 17.

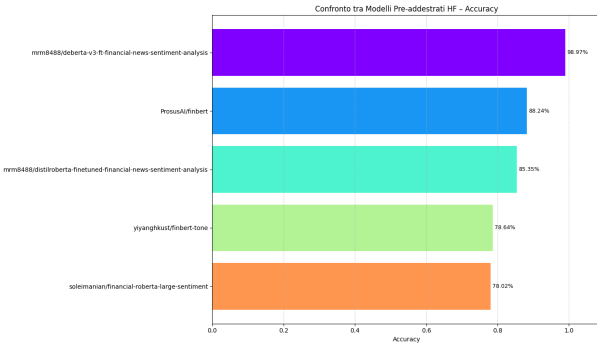


Figure 17: Accuracy dei modelli pre-addestrati su test set (senza fine-tuning)

Il miglior modello in questa fase è risultato essere `mrm8488/deberta-v3-ft-financial-news-sentiment-analysis`, ottenendo un'accuracy quasi perfetta del 98,97%.

Analisi degli errori. Come per i modelli baseline, è stata calcolata la matrice di confusione del miglior modello (Figura 18) per analizzare quali classi venivano confuse più frequentemente. Essa mostra un'elevata accuratezza su tutte le classi, con pochissimi errori. Le uniche confusioni si osservano in classe *neutral*, che tende a sovrapporsi leggermente con *positive*. Nessuna istanza di sentiment negativo è stata erroneamente classificata come positivo e viceversa, a conferma di una chiara distinzione tra polarità opposte.

9.2 Fine-Tuning supervisionato

Dopo l'inferenza zero-shot, si è proceduto a un **fine-tuning supervisionato** del miglior modello (`deberta-v3-ft`) sul dataset annotato. L'obiettivo era adattare ulteriormente il modello al dominio e al task specifico.

Strategia. Durante il fine-tuning:

- Tutti i layer interni del modello sono stati **congelati**, eccetto la *classification head*, per limitare l'overfitting e ridurre il tempo di addestramento.
- Il dataset è stato diviso in training e validation set (stratificato) e convertito in formato Dataset HuggingFace.
- È stato utilizzato Trainer con EarlyStopping e DataCollatorWithPadding.

Monitoraggio. L'andamento della loss (Figura 19) mostra un iniziale miglioramento su entrambe le curve. Tuttavia, dopo 6 epoche, grazie all'early stopping (pazienza 3), l'allenamento è terminato a causa del peggioramento della *validation loss* che nella maggior parte dei casi indica l'inizio dell'overfitting del modello (il modello si adatta troppo ai dati di training).

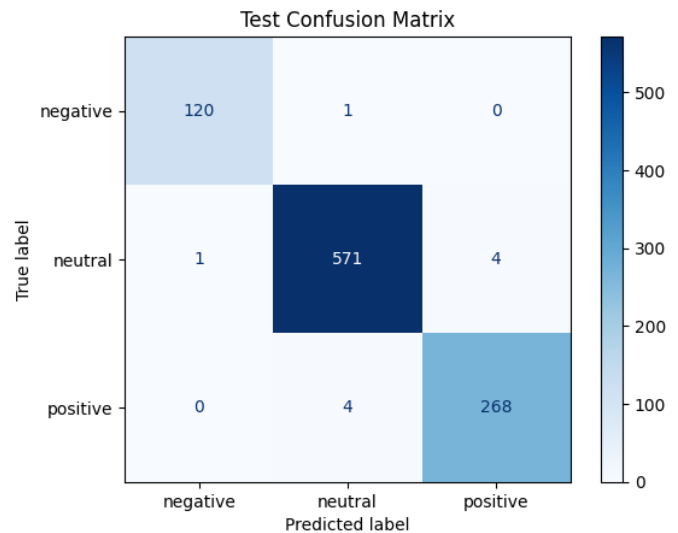


Figure 18: Matrice di confusione per il miglior modello pre-addestrato (fase inferenza)



Figure 19: Andamento della training e validation loss durante il fine-tuning

Analisi degli errori. La nuova matrice di confusione (Figura 20) mostra che il comportamento del modello è rimasto quasi invariato: infatti, il numero di predizioni corrette e di errori (10) sono gli stessi rispetto a prima, l'unica differenza è dove il modello sbaglia e predice correttamente.

Infine, sono stati analizzati manualmente alcuni esempi in cui il modello sbagliava la predizione. Questo ha permesso di evidenziare limiti legati all'ambiguità semantica, alla presenza di frasi neutre ma sintatticamente complesse, o a contesti finanziari altamente specialistici. La tabella 11 mostra un esempio di errore fatto dal modello:

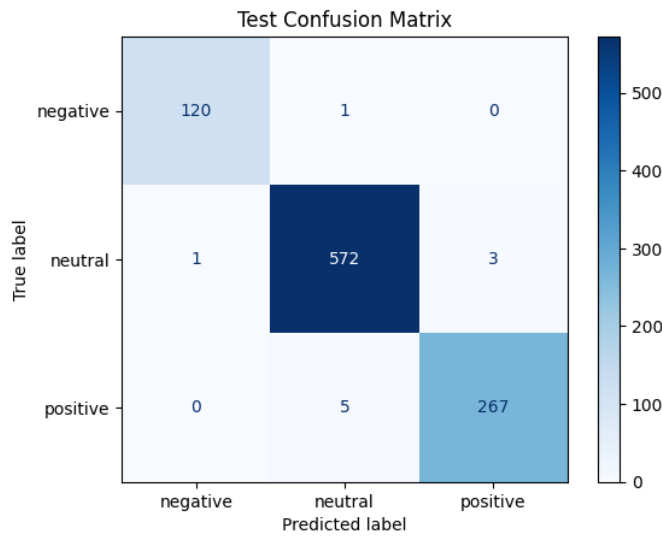


Figure 20: Matrice di confusione dopo il fine-tuning del modello pre-addestrato

Testo Originale	Label Reale	Label Predetta
So far the company has awarded more than \$350,000 worth of tools and materials.	Neutral	Positive
A maximum of 20 employees , who work in Karttakeskus and are responsible for producing Geographic Information Services , will be affected , the company added.	Neutral	Negative

Table 11: Esempi di classificazione errata da parte del modello fine-tunato

Osservazioni:

- **Esempio 1 – Falso positivo:** la frase riporta un'informazione neutra (assegnazione di strumenti), ma l'uso di parole come *awarded* e l'alto valore monetario (\$350,000) può aver indotto il modello ad associare un sentiment positivo. Questo suggerisce che il modello è sensibile a keyword numeriche e verbali che, in contesti non emotivi, possono comunque suonare "ottimiste".
- **Esempio 2 – Falso negativo:** la frase riguarda una riorganizzazione aziendale che impatta un numero limitato di dipendenti. Nonostante l'informazione sia descrittiva, il modello ha probabilmente interpretato termini come *affected* e *maximum of 20 employees* in chiave negativa, sottovalutando l'aspetto neutro e contestuale. Questo tipo di errore è frequente quando il modello non riesce a distinguere tra un rischio effettivo e una semplice descrizione operativa.

Questi esempi mostrano che, sebbene le metriche globali siano ottime, il modello può ancora confondere sfumature linguistiche non legate a un reale sentimento ma solo a parole "caricate" semanticamente.

9.2.1 Visualizzazione dello spazio degli embedding. Per comprendere meglio le ragioni alla base degli errori residui del modello, è stata effettuata una proiezione bidimensionale degli **embedding CLS** (token [CLS]), utilizzati dal modello come rappresentazione dell'intera frase per la classificazione.

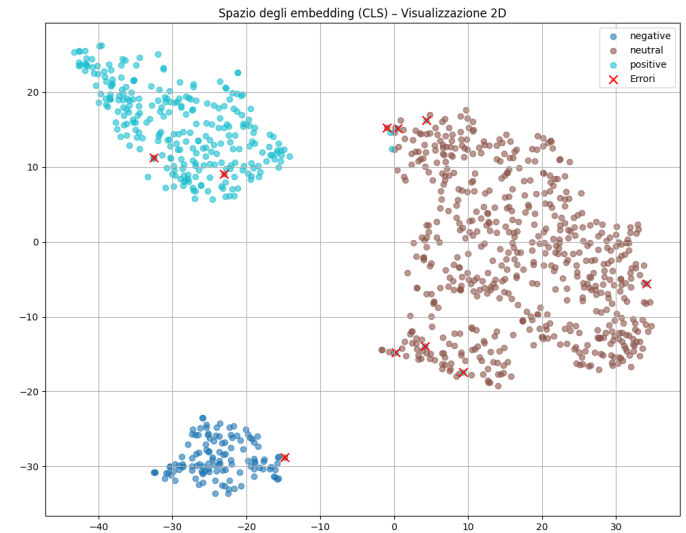


Figure 21: Proiezione 2D dello spazio degli embedding (CLS) con evidenza degli errori

Interpretazione. Come si osserva dalla Figura 21, ciascun punto rappresenta una frase del test set, codificata nello spazio latente dal modello pre-addestrato. I colori indicano la classe reale (positive, neutral, negative), mentre i simboli rossi ("X") evidenziano i campioni classificati erroneamente.

Distribuzione e sovrapposizione. La visualizzazione mostra tre principali cluster ben distinti, corrispondenti alle tre classi. Tuttavia, alcuni errori sono localizzati in regioni di **sovrapposizione semantica** tra due classi, dove gli embedding di frasi neutral si avvicinano molto a quelli di classe positiva o negativa. Ciò suggerisce che il modello non ha una separazione netta in quelle aree dello spazio e che **la rappresentazione semantica delle frasi è ambigua o meno definita.**

Questo tipo di analisi conferma che gli errori non sono casuali, ma avvengono in corrispondenza di aree del feature space dove la densità degli embedding di classi diverse si avvicina.

Conclusione. L'utilizzo di modelli pre-addestrati, in particolare DeBERTa-v3 fine-tuned su news finanziarie, ha dimostrato performance superiori rispetto ai modelli baseline. La possibilità di sfruttare direttamente il testo grezzo semplifica notevolmente il flusso di lavoro, ed elimina la necessità di creare embedding manuali o gestire feature ingegnerizzate come il topic. Il fine-tuning

ha ulteriormente migliorato le performance, consolidando questo approccio come il più promettente per futuri sviluppi.

Nota bene: i risultati riportati sono stati ottenuti utilizzando il dataset sottoposto a pre-processing leggero. La medesima pipeline è stata successivamente applicata anche al dataset con pre-processing aggressivo, ma in questo caso le prestazioni sono risultate inferiori. Pertanto, nel presente documento vengono mostrati esclusivamente i risultati relativi al pre-processing light.

10 EVALUATION

La valutazione dei modelli è stata condotta analizzando i risultati salvati automaticamente dalla pipeline di addestramento implementata in Apache Airflow. Ogni esperimento produce un file contenente le metriche di classificazione principali (accuracy, F1 per ciascuna classe), le informazioni sull'embedding utilizzato, la strategia di bilanciamento adottata, e un identificatore temporale. Tali dati sono stati aggregati nel file `comparison_table.csv`, poi analizzato tramite un notebook dedicato.

10.1 Confronto tra gli esperimenti

La Tabella 12 sintetizza i risultati ottenuti dai modelli testati con diverse configurazioni di embedding e bilanciamento.

Dai risultati emerge che il modello basato su SBERT senza bilanciamento ottiene la migliore accuracy complessiva (0.718), mentre la miglior performance sulle classi minoritarie (positive e negative) si registra con SBERT bilanciato, che raggiunge F1-score di 0.605 per la classe positiva e 0.658 per quella negativa.

Word2Vec, sebbene più leggero computazionalmente, mostra performance inferiori, in particolare sulla classe negativa. L'uso del bilanciamento migliora leggermente la stabilità delle performance su Word2Vec, ma penalizza fortemente l'accuracy globale.

10.2 Osservazioni sulla qualità delle predizioni

L'analisi dei modelli suggerisce che:

- L'uso di embedding semantici a livello di frase (SBERT) è significativamente più efficace rispetto all'approccio bag-of-words (Word2Vec).
- L'adozione di tecniche di bilanciamento, sebbene penalizzi leggermente l'accuracy, consente un miglioramento sostanziale nella **predizione delle classi minoritarie**, che è spesso un obiettivo prioritario in compiti di sentiment analysis.
- Tutti i modelli mostrano una certa difficoltà nel distinguere le frasi *positive*, come evidenziato dai F1-score più bassi su tale classe rispetto a *neutral* e *negative*.

10.3 Modello Selezionato

Il miglior modello selezionato è stato quello identificato dall'esperimento `sbert_unbalanced_20250623T145209`, salvato automaticamente nella cartella `best_model_latest` dalla pipeline `compare_and_update_best_model_dag`. Questo è il modello utilizzato nel sistema finale per le predizioni.

11 DEPLOYMENT E ARCHITETTURA OPERATIVA

Al termine della fase di valutazione, il sistema è stato progettato per selezionare e rendere disponibile automaticamente il miglior modello di classificazione del sentiment, attraverso una logica di deployment automatizzato. Questo è reso possibile dal DAG `compare_and_update_best_model_dag`, che esegue le seguenti operazioni:

- legge tutti i file CSV di confronto generati dagli esperimenti precedenti (`comparison_table.csv`);
- valuta i modelli sulla base dell'*accuracy* e delle F1-score per ciascuna classe;
- identifica il modello con le prestazioni migliori in termini di equilibrio tra le classi;
- aggiorna dinamicamente la directory `best_model_latest`, sovrascrivendo i file del modello precedente con quelli del nuovo modello selezionato (inclusi pesi, configurazione, embeddings e file di log).

Questa soluzione consente una gestione continua e tracciabile dell'evoluzione dei modelli, garantendo che la versione più performante sia sempre disponibile per un'eventuale fase di inferenza real-time o batch.

12 ESPOSIZIONE DEL MODELLO TRAMITE FASTAPI

Per consentire l'accesso al modello di sentiment analysis da parte di sistemi esterni (es. interfacce web, applicazioni client, strumenti di testing), è stato sviluppato un servizio RESTful basato su FastAPI. Questo servizio consente di interrogare il modello direttamente tramite chiamate HTTP POST, fornendo come input una o più frasi testuali e restituendo come output la predizione della classe di sentiment associata.

L'API espone un endpoint `/predict` che riceve in input un dizionario JSON con una chiave `sentences` contenente una lista di frasi da classificare. Il servizio carica il modello e il tokenizer associato al deployment più recente, precedentemente selezionato tramite il DAG `compare_and_update_best_model_dag`, e restituisce un output JSON con le predizioni effettuate.

Listing 1: Esempio di richiesta HTTP POST

```
POST /predict HTTP/1.1
Content-Type: application/json

{
  "sentences": [
    "The company's revenue increased significantly.",
    "The financial results were disappointing."
  ]
}
```


Embedding	Bilanciamento	Accuracy	F1 Neutral	F1 Positive	F1 Negative
SBERT (MiniLM)	Nessuno	0.718	0.802	0.556	0.631
SBERT (MiniLM)	Class weight	0.686	0.745	0.605	0.658
Word2Vec (GloVe)	Nessuno	0.672	0.784	0.464	0.489
Word2Vec (GloVe)	Class weight	0.589	0.694	0.479	0.480

Table 12: Prestazioni dei modelli di classificazione in funzione di embedding e strategia di bilanciamento.

Listing 2: Esempio di risposta del modello

```
{
  "predictions": [
    "positive",
    "negative"
  ],
  "timestamp": "2025-06-24 14:11:42"
}
```

L'applicazione FastAPI è contenuta in un modulo Python indipendente (app.py) e può essere avviata tramite uvicorn. Il caricamento del modello è dinamico e punta alla cartella `best_model_latest`, aggiornata periodicamente dai DAG di Airflow. L'infrastruttura API è pensata per un utilizzo leggero e rapido in fase di test, ma può essere estesa con funzionalità aggiuntive come logging, autenticazione o versionamento.

La risposta include inoltre un timestamp utile per il tracciamento delle richieste, in ottica di audit o future analisi su errori o misclassificazioni.

Architettura degli ambienti e gestione del ciclo di vita

Per garantire un'organizzazione modulare tra sperimentazione interattiva e orchestrazione automatizzata, sono stati predisposti due ambienti distinti all'interno dell'IDE PyCharm:

- **Ambiente locale** (eda.env): un ambiente virtuale Python basato su venv, dedicato esclusivamente all'esplorazione interattiva dei dati tramite Jupyter Notebook. Le principali librerie installate includono pandas, seaborn, matplotlib, wordcloud e nltk.
- **Ambiente containerizzato**: è stato predisposto un ambiente multi-container tramite Docker Compose, che orchestralmente coordina i servizi necessari per il funzionamento di Apache Airflow con esecuzione asincrona basata su CeleryExecutor. La configurazione include i seguenti componenti:
 - airflow-webserver, airflow-scheduler, airflow-worker, airflow-triggerer, airflow-init, per gestire l'interfaccia utente, la schedulazione, l'esecuzione parallela dei DAG e l'inizializzazione del sistema;
 - Redis (versione 7.2-bookworm) come message broker per la comunicazione asincrona tra i worker;
 - PostgreSQL (versione 13) come backend per la persistenza dei metadati Airflow;

- configurazione del mount di directory locali (./dags, ./logs, ./data, ./models, ecc.) all'interno dei container, per garantire sincronizzazione e tracciabilità dei file;
- installazione automatica dei pacchetti Python necessari all'avvio tramite comando `pip install -r /requirements.txt` eseguito all'interno dei container webserver, scheduler e worker.

L'infrastruttura segue una configurazione basata su build di immagine personalizzata, evitando l'uso improprio di `_PIP_ADDITIONAL_REQUIREMENTS` per l'installazione dinamica dei pacchetti a ogni avvio.

Questa separazione consente:

- di mantenere snello l'ambiente Airflow, migliorando i tempi di build e di avvio dei container;
- di evitare conflitti di dipendenze tra librerie analitiche e operative;
- di garantire riproducibilità e scalabilità secondo i principi MLOps.

Infine, PyCharm è stato configurato con due interpreti Python distinti: uno per l'ambiente locale (EDA) e uno per lo sviluppo e il test delle pipeline in Docker. Questo approccio ha permesso di integrare in modo efficace sperimentazione e automazione, massimizzando la tracciabilità e l'efficienza lungo tutto il ciclo di vita del progetto.

13 CONCLUSIONI

Il progetto *FinSent* ha dimostrato l'efficacia dell'integrazione tra tecniche avanzate di Natural Language Processing e principi di ingegneria MLOps per la sentiment analysis nel dominio finanziario. Attraverso la costruzione di una pipeline automatizzata e modulare, orchestrata con Apache Airflow, è stato possibile affrontare in modo sistematico le fasi di preprocessing, embedding, addestramento, valutazione e deployment dei modelli.

L'adozione di modelli transformer pre-addestrati, in particolare FinBERT e DeBERTa-v3-ft, ha evidenziato un netto miglioramento delle performance rispetto ai modelli baseline, raggiungendo un'accuratezza prossima al 99% nel caso del fine-tuning supervisionato. Al contempo, l'impiego di soluzioni più leggere come SBERT (MiniLM) per la pipeline automatizzata, ha garantito un buon compromesso tra accuratezza ed efficienza computazionale, rendendo il sistema adatto all'integrazione in ambienti di produzione.

L'analisi tematica tramite BERTopic ha ulteriormente arricchito la comprensione del dataset, evidenziando la relazione tra contenuti semantici e polarità emotiva. L'infrastruttura sviluppata consente

inoltre una gestione automatizzata del ciclo di vita dei modelli, inclusi aggiornamento, monitoraggio delle performance e esposizione tramite API RESTful.

Nonostante i risultati promettenti, alcune criticità permangono, in particolare la difficoltà nel discriminare frasi ambigue o sfumate, nonché lo sbilanciamento delle classi che penalizza la predizione delle etichette minoritarie.

In conclusione, *FinSent* rappresenta un contributo concreto alla progettazione di sistemi intelligenti per l'analisi automatica delle notizie economiche, combinando robustezza ingegneristica, flessibilità sperimentale e capacità predittiva, con potenziali applicazioni in contesti real-time di supporto alle decisioni finanziarie.

14 SVILUPPI FUTURI

Tra le prospettive evolutive del progetto si segnala l'estensione dell'architettura verso un sistema **MLOps di livello 2**. Questo avanzamento includerebbe:

- Monitoraggio in produzione. Sebbene il sistema attuale implementi la selezione automatica del miglior modello basata su metriche offline, non è ancora previsto un sistema di monitoraggio continuo post-deployment.
- Integrazione di strumenti di osservabilità come MLflow o Evidently AI, al fine di:
 - monitorare l'evoluzione delle metriche in produzione (accuracy, F1, ecc.);
 - rilevare fenomeni di data drift e concept drift;
 - attivare retraining condizionato al degrado delle performance;
 - implementare notifiche automatiche o visualizzazioni tramite Grafana o Prometheus.

Tali funzionalità sono fondamentali per il passaggio da un MLOps di livello 1 a un livello 2 pienamente maturo.

- riaddestramento automatico del modello al superamento di soglie predefinite;
- implementazione di test di regressione sui modelli prima del deployment;
- gestione centralizzata e versionata di esperimenti, configurazioni e metriche;
- integrazione con pipeline CI/CD complete per modelli e dati.

Sul piano modellistico, si propone l'esplorazione delle seguenti direzioni:

- utilizzo di tecniche di data augmentation controllata per riequilibrare le classi;
- adozione di modelli instruction-tuned o prompt-based, per una gestione più efficace di frasi ambigue;
- estensione del sistema a livello multilingua e multi-mercato;
- integrazione di knowledge graph o approcci RAG (Retrieval-Augmented Generation) per analisi documentali più profonde.

Queste evoluzioni renderebbero il sistema ancora più adattabile, scalabile e orientato all'impiego in ambienti produttivi complessi e dinamici.

REFERENCES

[1] "LexisNexis," <https://www.lexisnexis.com/en-us/gateway.page>.

[2] "OMX_{Helsinki}," .

P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "eng-Good debt or bad debt: Detecting semantic orientations in economic texts," *engJournal of the Association for Information Science and Technology*, vol. 65, no. 4, pp. 782–796, 2014.